Atividade 4 - Projeto e Arquitetura de Software

MC656 - Engenharia de Software

Caio Lima Albuquerque - RA: 288808
Giovana Jacome Marchetti - RA: 177700
Julia de Souza Nardo - RA: 281272
Leonardo Carvalho De Luca - RA: 288820
Samuel Rodrigues Ferreira - RA: 195440

26 de outubro de 2026

1 Introdução

Este documento apresenta a documentação arquitetural e a evolução do design do sistema de votação em três contextos (Eleição Preferencial Australiana, Assembleia do CACo e *Blood on the Clocktower* (BotC)), consolidando a entrega da Atividade 4. O trabalho contempla o mapeamento da arquitetura em nível de componentes (C4 Nível 3), as decisões arquiteturais tomadas na transição para a interface Web com FastAPI, a aplicação de dois padrões de projeto GoF para solucionar problemas reais de acoplamento e criação de objetos, e a análise e refatoração de *code smells* com auxílio de análise estática (SonarCloud e Ruff), preservando integralmente o comportamento verificado pela suíte de testes automatizados.

Rastreabilidade e Versão Avaliada no GitHub:

- **Repositório:** `https://github.com/GiizAzul/MC656`
- **Tag da Versão Avaliada:** `a4-v1.0.0`
- **Filtro de Issues e PRs:** Todas as issues e Pull Requests desta entrega estão identificados com a label `Atividade4`.

2 Arquitetura do Sistema

2.1 Diagrama C4 — Nível de Componentes (Nível 3)

A Figura 1 apresenta o diagrama C4 em nível de componentes (Nível 3) do contêiner da Aplicação Web/API construída sobre o framework *FastAPI*, detalhando como os controladores HTTP, dependências transversais, serviços de segurança e motores de domínio interagem entre si.

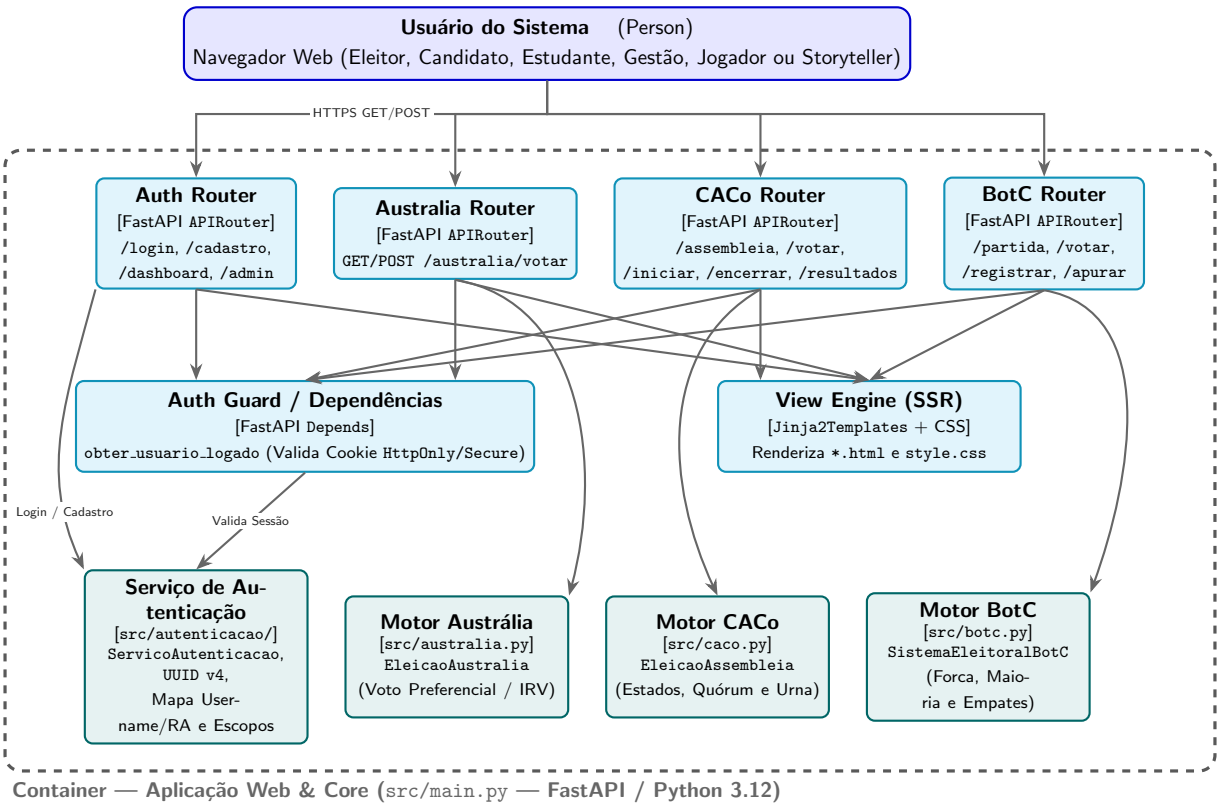


Figura 1: Diagrama C4 em Nível de Componentes (Nível 3) da Aplicação Web.

2.2 Principais Componentes, Responsabilidades e Relações

A arquitetura interna do sistema é modularizada para separar os protocolos de interface das regras matemáticas de apuração e controle de elegibilidade. A Tabela 1 descreve cada componente presente na implementação atual (‘src/’).

Componente	Arquivos (`src/`)	Responsabilidades e Relações
Bootstrap & State	`main.py`	Ponto de entrada da aplicação FastAPI. Inicializa as instâncias singleton em memória (`app.state`), monta os arquivos estáticos (`/static`), registra os roteadores e intercepta exceções globais de sessão.
Auth Router	`routers/auth_router.py`	Gerencia autenticação (`/login`, `/logout`), criação de contas (`/cadastro`) e renderização dos painéis (`/dashboard`, `/admin`). Consome o *Serviço de Autenticação* e atualiza a lista de eleitores ativos.

Australia Router	`routers/australia_router.py`	Controla a exibição da cédula preferencial (`GET /australia/votar`) e processa o envio do ranking (`POST /australia/votar`), validando sequencialidade (1 a N) e unicidade de voto por eleitor antes de repassar ao motor de domínio.
CACo Router	`routers/caco_router.py`	Gerencia o fluxo da assembleia estudantil (`/caco/assembleia`, `/votar`, `/iniciar`, `/encerrar`, `/resultados`). Restringe comandos de condução ao escopo `CACO_GESTAO` e interage com o motor `EleicaoAssembleia`.
BotC Router	`routers/botc_router.py`	Expõe as rotas da partida (`/botc/partida`, `/votar`) e ações exclusivas do Mestre (`/registrar`, `/apurar`), delegando a lógica ao `SistemaEleitoralBotC`.
Middleware / Auth Guard	`routers/dependencias.py`	Componente transversal injetado via `Depends` (`obter_usuario_logado`). Extrai o cookie `sessao_usuario`, resolve o objeto `Usuario` no banco em memória e dispara `RequerRedirecionamento Exception` em acessos não autenticados.
View Engine (SSR)	`templates/*.html` `static/style.css`	Camada de apresentação baseada em templates *Jinja2* com herança de layout (`base.html`) e estilização centralizada via CSS semântico.

Serviço de Autenticação e Autorização	`autenticacao/base.py`
	`autenticacao/autorizacao.py`
	
	`autenticacao/caco.py`
	`autenticacao/botc.py`
	`autenticacao/australia.py`
		Mantém o repositório de contas indexado por `UUID v4` (`_banco_por_id`) e mapeamento duplo por `username` e `RA` (`_mapa_usernames`). Define a hierarquia polimórfica de usuários e valida permissões por escopo (`MACRO_SCOPES`).

Motores de Domínio Eleitoral	`australia.py`
	`caco.py`
	`botc.py`
	`interfaces.py`
		Núcleo de regras de negócio puras, independentes de I/O: `EleicaoAustralia` (transferência iterativa de votos e eliminação), `EleicaoAssembleia` (máquina de estados, quórum de 10% e contagem anônima) e `SistemaEleitoralBotC` (regras de maioria na forca e empate).

Tabela 1: Principais componentes da arquitetura e suas responsabilidades.

2.3 Estilos Arquiteturais Adotados

A estrutura da aplicação combina dois estilos arquiteturais complementares:

1. **Arquitetura em Camadas (*Layered Architecture*) com padrão MVC Server-Side Rendered (SSR):** A aplicação separa claramente a **Camada de Apresentação** (Templates Jinja2 + CSS), a **Camada de Controle e Roteamento HTTP** (Roteadores FastAPI e Guardas de Dependência) e a **Camada de Domínio e Serviços** (‘src/australia.py‘, ‘src/caco.py‘, ‘src/botc.py‘, ‘src/autenticacao/‘). As dependências fluem estritamente de cima para baixo: os motores de domínio não conhecem objetos HTTP (‘Request‘, ‘Form‘, ‘Cookie‘) nem comandos de terminal (‘input‘/‘print‘), o que permitiu manter a compatibilidade simultânea entre a antiga interface CLI e a nova interface Web.

2. **Arquitetura Modular Orientada a Domínios (*Vertical Slices* nos Controladores):** Na camada de roteamento e de visualização, os componentes são particionados verticalmente pelos três contextos de negócio (`australia_router`, `caco_router`, `botc_router`) apoiados por um módulo transversal de autenticação e autorização (`auth_router` e `dependencias`).

2.4 Decisões Arquiteturais Relevantes (ADRs)

Decisão 1: Desacoplamento do Domínio e Coexistência de Múltiplas Interfaces (Web SSR com FastAPI/Jinja2 + CLI)

- **Contexto e Justificativa:** Na Atividade 2, o sistema operava exclusivamente via terminal (CLI). Com a elicitação de requisitos da Atividade 3, surgiu a necessidade de usabilidade para usuários leigos via navegador (Web). Em vez de reescrever as regras de negócio ou criar uma SPA complexa em JavaScript separado (React/Vue), a equipe decidiu manter os motores de domínio puros em Python e introduzir o *FastAPI* acoplado ao motor de templates *Jinja2* (Server-Side Rendering).
- **Consequências e Trade-offs:**
  - *Positivas:* Reaproveitamento integral (100%) dos testes de unidade de domínio já existentes; simplicidade de deploy em um único processo Python; ausência de problemas de CORS; e facilidade de testar o fluxo completo de ponta a ponta usando o `TestClient` do Starlette/FastAPI (atingindo 100% de cobertura nos roteadores).
  - *Negativas / Trade-offs:* A renderização no servidor exige recarregamento de página a cada submissão de formulário (`POST` → `TemplateResponse/RedirectResponse`), oferecendo menor interatividade em tempo real do que uma arquitetura baseada em WebSockets ou SPA para atualizações automáticas de placar.

Decisão 2: Gestão de Estado Centralizada em Memória (`app.state`) e Sessão via Cookies `HttpOnly` com Injeção de Dependências

- **Contexto e Justificativa:** O sistema precisava garantir persistência de sessão entre múltiplas votações, impedir votos duplicados e isolar os escopos de permissão (Eleitor, Candidato, Estudante, Gestão, Jogador e Storyteller) sem introduzir a sobrecarga de configuração de um SGBD relacional externo nesta etapa do projeto. Decidiu-se centralizar o estado no objeto `app.state` do FastAPI, indexar usuários internamente por `UUID v4` imutável (com índice secundário por `username` e `RA`) e gerenciar a sessão por meio de cookies `HttpOnly` validados por injeção de dependência (`Annotated[Usuario, Depends(obter_usuario_logado)]`).
- **Consequências e Trade-offs:**
  - *Positivas:* Proteção contra acesso a cookies via scripts maliciosos no navegador (`httponly=True`); padronização da verificação de autenticação fora do corpo das rotas (evitando código morto e duplicação); altíssima velocidade de execução dos 180 testes automatizados sem necessidade de *containers* de banco de dados.
  - *Negativas / Trade-offs:* Como o estado reside na memória RAM do processo Uvicorn (`app.state`), o reinício do servidor apaga os votos e cadastros dinâmicos, e a aplicação não pode ser escalada horizontalmente com múltiplos *workers* sem antes migrar o estado para um armazenamento compartilhado (como Redis ou PostgreSQL).

3 Padrões de Projeto

Para evoluir o design interno da aplicação e reduzir o acoplamento identificado durante a expansão para a Web, foram selecionados e implementados dois padrões de projeto clássicos (GoF) voltados a problemas reais do código.

3.1 Aplicação 1: Padrão Criacional — *Factory Method / Simple Factory* na Criação Polimórfica de Usuários

Problema Identificado	Violação do Princípio Aberto-Fechado (OCP). O controlador Web (`src/routers/auth_router.py`, na função `processar_cadastro`) possuía cadeias extensas de `if/elif/else` verificando strings (`"CACO_ESTUDANTE"`, `"CACO_GESTAO"`, `"AUSTRALIA_ELEITOR"`, etc.), validando regras específicas (como a obrigatoriedade numérica do RA para estudantes) e instanciando diretamente as 6 subclasses concretas de `Usuario`.
Padrão Escolhido e Justificativa	**Simple Factory / Factory Method** (`UsuarioFactory`). A criação polimórfica de contas foi encapsulada em uma fábrica dedicada na camada de autenticação (`src/autenticacao/`). O padrão é ideal porque centraliza as regras de validação de parâmetros de criação (como conversão e checagem do RA) e desacopla os controladores Web das classes concretas de cada domínio eleitoral.
Consequências da Solução	• **Positivas:** Redução drástica da Complexidade Cognitiva na rota `POST /cadastro`; facilidade para adicionar novos perfis de usuário em um único ponto do sistema. • **Negativas:** Introdução de mais uma classe/estrutura na camada de autenticação.
Rastreabilidade no GitHub	• **Issue:** Refatoração da instanciação de usuários com o padrão Simple Factory #94 • **Pull Request:** Refactor: Implementar padrão Simple Factory no cadastro de usuários #95

Tabela 2: Especificação da primeira aplicação de padrão de projeto.

Problema Identificado	No módulo `src/caco.py` (`EleicaoAssembleia`), o controle do ciclo de votação utilizava o atributo `estado`, representado anteriormente por valores de um `Enum`, além de verificações condicionais para controlar as operações de início, registro de votos e encerramento. Parte das regras de controle também era distribuída entre a classe de domínio e o roteador, dificultando a centralização das transições de estado e a manutenção do código.
Padrão Escolhido e Justificativa	**Padrão State**. Foram criadas classes para representar os estados de espera, votação em andamento e votação encerrada, definidas em `src/estados.py`. A abstração `SistemaEleitoral`, em `src/interfaces.py`, passou a delegar as operações de início, registro de votos e encerramento ao estado atual. A implementação de `EleicaoAssembleia` foi adaptada para utilizar essa estrutura, e o roteador do CACo foi atualizado para trabalhar com a nova representação dos estados. Dessa forma, os comportamentos e as transições da votação passam a ser organizados de acordo com o estado ativo.
Consequências da Solução	• **Positivas:** Encapsulamento dos comportamentos associados a cada estado; centralização das transições da votação; redução do acoplamento entre a lógica de domínio e o roteador; maior facilidade de manutenção e extensão para os demais contextos; possibilidade de rejeitar operações incompatíveis com o estado atual por meio de exceções `RuntimeError`. • *Negativas:* Aumento do número de classes para representar os estados e necessidade de adaptar a interface `SistemaEleitoral` e os componentes dependentes da representação anterior.
Rastreabilidade no GitHub	• **Issue:** Refatorar o controle de estados da votação utilizando o padrão State #91 • **Pull Request:** Refatora o controle de estados da votação do CACo com o padrão State #92

Tabela 3: Especificação da segunda aplicação de padrão de projeto.

4 Code Smells e Refatoração

Com o auxílio da ferramenta de análise estática contínua **SonarCloud** integrada ao repositório e do linter **Ruff**, o código foi inspecionado em busca de *code smells* e *design smells*. A seguir, documentamos duas ocorrências de tipos distintos identificadas no sistema.

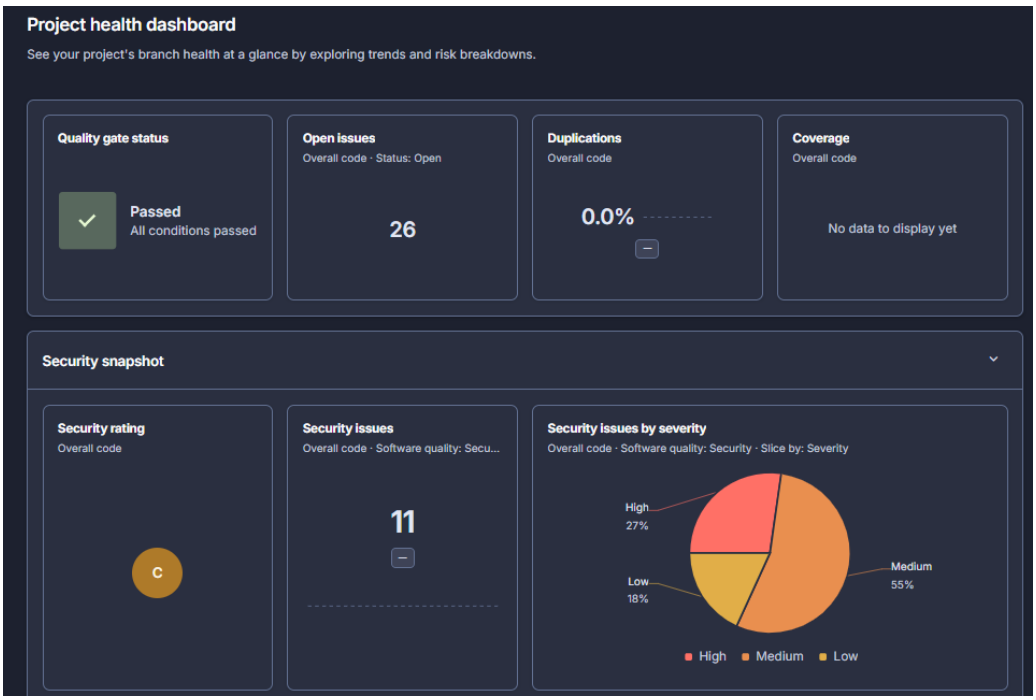


Figura 2: Ilustração do dashboard do SonarCloud

4.1 Ocorrência 1: Controles de Formulário Desacoplados de Rótulos Semânticos (`Web:S6853` e `Web:InputWithoutLabelCheck`)

- **Localização:** `src/templates/cadastro.html`, `src/templates/admin.html` e `src/templates/australia.html`.
- **Smell Identificado: Ausência de associação explícita entre rótulos (`<label>`) e controles de entrada (`<input>`, `<select>`)** — Regras SonarCloud `Web:S6853` (*Label elements should have a text label and an associated control*) e `Web:InputWithoutLabelCheck` (*Input, select and textarea tags should be labeled*).
- **Justificativa:** Nos templates Jinja2 da camada de visualização, as tags `<label>` foram declaradas apenas visualmente acima dos campos de entrada sem o atributo `for`, e os elementos `<input>` e `<select>` não possuíam identificadores únicos (`id`). Na tela de votação da Austrália (`australia.html`), os nomes dos candidatos estavam encapsulados em tags genéricas `<span>` ao lado dos inputs numéricos de ranqueamento.
- **Impacto na Manutenção e Evolução:** Compromete diretamente os requisitos não funcionais de **Usabilidade e Acessibilidade** do sistema (tecnologias assistivas e leitores de tela não conseguem identificar qual dado deve ser inserido em cada campo), reduz a área clicável de foco dos formulários em dispositivos móveis e dificulta a seleção determinística de elementos DOM em futuros testes automatizados de interface (*End-to-End*).

4.2 Ocorrência 2: Cookie de Sessão sem Diretiva de Transporte Seguro (`python:S2092`)

- **Localização:** `src/routers/auth_router.py` (função `processar_login`).
- **Smell / Hotspot Identificado: Criação de Cookie de Autenticação sem a flag `secure`** (Regra SonarCloud `python:S2092`).

- **Justificativa:** Nas rotas de login e cadastro, o cookie `sessao_usuario` era emitido apenas com `httponly=True`, omitindo os atributos `secure=True` e `samesite="lax"`, mas se um cookie de sessão for enviado por uma conexão HTTP não criptografada, um invasor que conseguir interceptar o tráfego pode roubá-lo. Com um cookie de sessão válido, o invasor pode se passar pela vítima e ter acesso completo à conta dela sem precisar saber a senha. Mesmo em sites que usam principalmente HTTPS, uma única requisição HTTP contendo o cookie de sessão já é suficiente para expô-lo.

- **Impacto na Manutenção e Evolução:** A ausência da flag `secure` expõe o token de sessão a interceptações em conexões não criptografadas (*Man-in-the-Middle*).

4.3 Refatoração Realizada

- **Ocorrências Refatoradas:** Ambas as ocorrências apontadas pelo SonarCloud e pelo Ruff foram integralmente tratadas para zerar os alertas de qualidade do projeto:

  1. **Refatoração Semântica dos Templates HTML:** Nos arquivos `cadastro.html` e `admin.html`, todos os elementos `<input>` e `<select>` receberam atributos `id` semânticos vinculados aos respectivos `<label for="...>`. Em `australia.html`, as tags `<span>` foram convertidas em `<label for="posicao_{{ loop.index }}>` pareadas dinamicamente com cada campo numérico da cédula.
  2. **Endurecimento de Sessão e Tipagem de Dependências:** Nos arquivos `auth_router.py`, a emissão do cookie de sessão foi refatorada para incluir `secure=True` e `samesite="lax"`.

- **Preservação do Comportamento:** A suíte completa de **180 testes automatizados** (`pytest`) foi executada após as alterações (ajustando o cliente de testes web para operar sobre `https://testserver`), mantendo 100% de aprovação e 100% de cobertura nos roteadores, com zero erros reportados pelo Ruff e pelo SonarCloud.

- **Rastreabilidade no GitHub:**

  - **Issue da Refatoração:** Correção de code smells de acessibilidade e segurança detectados pelo SonarCloud e Ruff #86
  - **Pull Request da Refatoração:** Refactor: Eliminar code smells do SonarCloud e Ruff #88